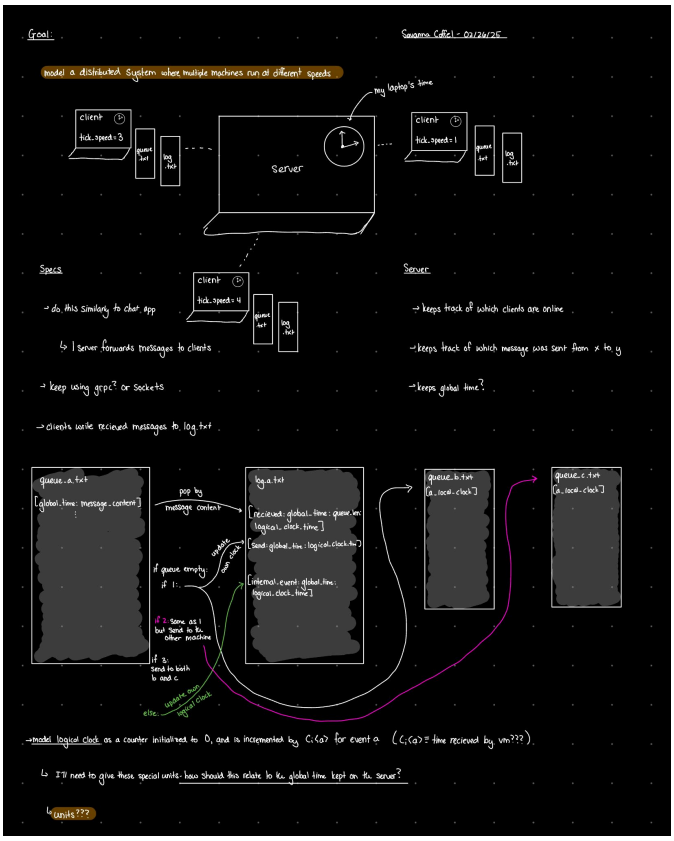
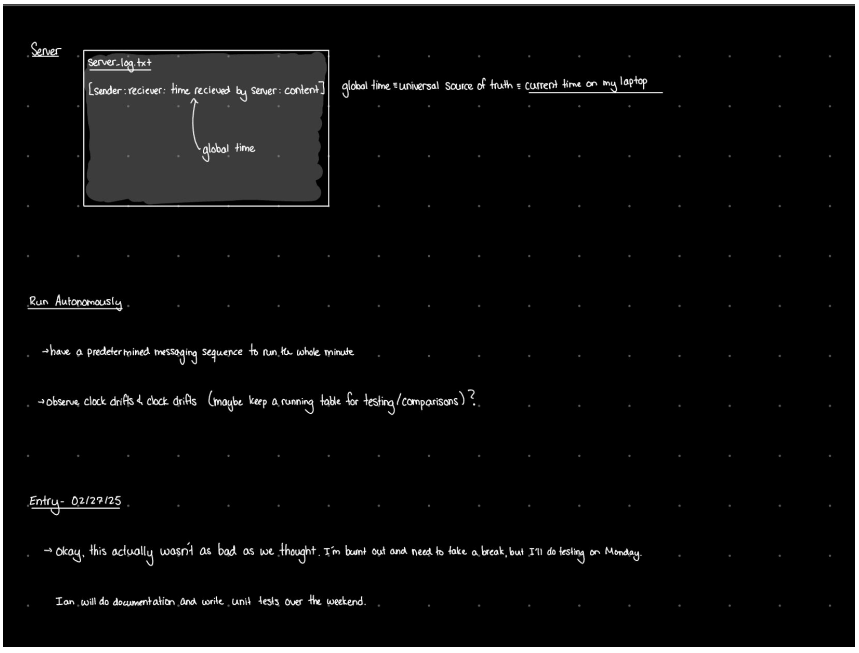
Engineering Notebook - Savanna Coffel and Ian Moore: DE2

Design Specs

I designed this similarly to Design Exercise 1. Now, I can simulate three clients and have a server pass messages between them by forwarding messages, and reuse much of the same design from before:



A notable difference is that now, each “client” has its own logical clock to update, and the server keeps a global time. I do not consider the server a virtual machine, only the clients.



Testing Results (Data from 3 March 2025)

Setup:

- Preloaded a couple of messages in the queue to be randomly sent off through the simulation, ensuring a nonempty queue.
- Continued incrementing the machine clock time to track incoming events relative to the machine.

Trial Outcomes:

Trial 1:

- Clock rate of a: 1 tick/s
- Clock rate of b: 3 ticks/s
- Clock rate of c: 6 ticks/s

Trial 2:

- Clock rate of a: 1 tick/s
- Clock rate of b: 5 ticks/s
- Clock rate of c: 2 ticks/s

Trial 3:

- Clock rate of a: 1 tick/s
- Clock rate of b: 6 ticks/s
- Clock rate of c: 2 ticks/s

Trial 4:

- Clock rate of a: 3 ticks/s
- Clock rate of b: 5 ticks/s
- Clock rate of c: 6 ticks/s

Trial 5:

- Clock rate of a: 4 ticks/s
- Clock rate of b: 1 tick/s
- Clock rate of c: 5 ticks/s

Overall Thoughts:

- **Size of Jumps in Values for Logical Clocks:** Towards the start of each simulation, we see increments of 1, but as the simulation continues, the clocks start to jump by 2s occasionally, and towards the end of the simulation, we even see some jumps in 3s.
- **Drift of the Clocks:** These clocks have closer rates as compared to some of the other trials. We'll definitely see clock drift, especially when grabbing messages from the queue repeatedly. The longer the simulation runs, and the more drastic the differences between the clock rates, the more clock drift we're likely to see.
- **Impact of Different Timings on Gaps in the Clock Values and Length of Message Queue:** When we pull messages from the queue repeatedly, the logical clocks get updated relatively linearly, so we're not very likely to see a clock gap here. If the message queue is empty, or if we're alternating between a length of 0 and a length of 1, we might see some variance for some of the incoming messages. Again, we won't see as drastic of a gap since the clock rates only differ by 1.

Additional Findings on Clock Cycle Variation:

- **Differences with Smaller Clock Cycle Variation and Smaller Probability of Internal Event:** If I change the probability of internal event generation by only having an internal event generated for 3/10 numbers, it seems like the clocks tend to drift more. This would make sense because of the time discrepancy between reading messages from the queue and receiving incoming messages, both of which would increment the logical clock compared to a single internal event. So this is not unbelievable.
- If we have smaller variations in the clock cycles, the clocks seem to drift less. This also makes sense, because there are less repeated incrementations of the logical clocks when events are relatively synchronized.
- We decided to keep tracking the internal machine clock throughout all the simulations, so we could see if there were any drastic changes here. It seems like this stays relatively consistent, incrementing across all trials, which we found interesting! (Note: if I had to do this again, I would maybe not design it this way - it's a little confusing to read from the logs.)

Notes on Testing:

We worked very hard to achieve a high level of test coverage to ensure the quality of our codebase. We wrote comprehensive unit tests for both the client and server. Our test cases are numerous and spread over several different files as we worked to increase coverage.

Our test coverage for the client after diligent work is 78%. The reason it is not higher is because some of the code only runs during socket failures, which we have difficulty simulating in testing, and some of the code is simply driver code. We have a detailed breakdown of the untested lines and why they are difficult to test in `README.md`

Our test coverage for the server after our best efforts is 91%. The reason it is not higher is because some of the exceptions are very rare and difficult to trigger, require simulating network failures which we are not prepared to do in our unit tests, or are entry points/driver code. Details are in our `README.md`

Overall we find that after diligent work on our test suite we are very satisfied with our efforts to ensure quality control.